

TRD

Technical Requirements Document (TRD)

AWS Glue Data Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue-based data pipeline. It defines the technical architecture, data structures, transformation logic, and operational parameters required to build a production-grade ETL solution that ingests customer and order data, performs data quality operations, implements SCD Type 2 using Apache Hudi, and generates analytical aggregates.

Scope

- In Scope:
 - Technical specifications for AWS Glue Spark jobs
 - Data schema definitions for source and target datasets
 - Apache Hudi configuration for SCD Type 2 implementation
 - AWS Glue Data Catalog integration specifications
 - S3 data lake storage patterns and paths
 - Data quality validation logic
 - Join and aggregation transformation specifications
 - Error handling and logging mechanisms
 - Runtime parameter definitions
- Out of Scope:
 - Infrastructure provisioning (IAM roles, VPC, security groups)
 - CI/CD pipeline implementation
 - Cost optimization tuning
 - Performance benchmarking
 - Data retention and archival automation
 - Monitoring dashboard creation
 - Alerting and notification setup

Assumptions

1. AWS Glue version 3.0 or higher with Spark 3.1+ and Python 3.7+

2. Apache Hudi library version 0.10.0 or higher is available in Glue environment
3. CSV files use comma delimiter, UTF-8 encoding, and include header row
4. Date fields in order data are in ISO 8601 format (YYYY-MM-DD) or parseable string format
5. Numeric fields (PricePerUnit, Qty) are parseable as decimal/double types
6. CustId is a string or integer type serving as primary/foreign key
7. OrderId is a string or integer type serving as primary key
8. Glue job runs with sufficient DPU allocation (minimum 2 DPUs recommended)
9. S3 bucket adif-sdlc has versioning enabled for data protection
10. Glue database gen_ai_poc_databrickscoe exists or will be created on first run
11. Hudi table primary key for ordersummary is composite: CustId + OrderId
12. SCD2 change detection compares all business columns (excluding IsActive, StartDate, EndDate, OpTs)
13. Incremental processing uses full dataset comparison (no CDC mechanism specified)
14. TotalAmount calculation: SUM(PricePerUnit Qty) per customer
15. OpTs uses job execution timestamp in UTC
16. StartDate and EndDate use date type (YYYY-MM-DD format)
17. IsActive is boolean type
18. Null string detection is case-sensitive for "Null"
19. Duplicate detection uses all columns with exact match logic
20. Glue job has IAM permissions for s3:GetObject, s3:PutObject, glue:CreateTable, glue:UpdateTable, glue:GetTable

•

2. Technical Requirements

TR-INGEST-001: Customer Data Ingestion from S3

- Related Functional Requirement ID(s): FR-INGEST-001
- Objective:

Read customer CSV files from S3 source location into a Spark DataFrame with inferred or explicit schema for downstream processing.

- Source Details:
- Source Type: S3 CSV Files
- Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/
- File Format: CSV
- Delimiter: Comma (,)

- Header: Present (first row)
- Encoding: UTF-8
- Target Details:
- Target Type: In-memory Spark DataFrame
- DataFrame Name: df_customer_raw
- Input Schema:

	Column Name	Data Type	Nullable	Notes	
	-----	-----	-----	-----	
	CustId	String	No	Primary key, customer identifier	
	Name	String	Yes	Customer full name	
	EmailId	String	Yes	Customer email address	
	Region	String	Yes	Geographic region	

- Output Schema:

Same as Input Schema (pass-through at this stage)

- Processing / Transformation Logic:
 1. Initialize Spark session with Glue context
 2. Use Glue DynamicFrame or Spark DataFrame reader to load CSV files
 3. Specify options:
 - format: "csv"
 - header: "true"
 - inferSchema: "true" (or explicit schema definition)
 - path: s3://adif-sdlc/sdlc_wizard/customerdata/
 4. Load all CSV files matching pattern .csv in the directory
 5. Store result in df_customer_raw DataFrame
 6. Log record count: df_customer_raw.count()
- Validation Rules:
 - Verify S3 path exists and is accessible
 - Verify at least one CSV file is present
 - Verify header row contains expected column names: CustId, Name, EmailId, Region
 - Verify record count > 0 after load
- Error Handling and Logging:
 - If S3 path does not exist: Log ERROR "Source path s3://adif-sdlc/sdlc_wizard/customerdata/ not found" and raise exception

- If no CSV files found: Log ERROR "No CSV files found in source path" and raise exception
- If schema mismatch: Log WARNING "Schema mismatch detected, attempting best-effort column mapping" and continue
- If load fails: Log ERROR with exception details and raise exception
- On success: Log INFO "Customer data loaded successfully: {record_count} records"
- Runtime Parameters:
 - source_customer_path: s3://adif-sdlc/sdlc_wizard/customerdata/ (configurable via Glue job parameter)
- Operational Notes:
 - Use Glue's pushdown predicate optimization if filtering is needed
 - Consider partition pruning if source data is partitioned (not specified in FRD)
 - Monitor CloudWatch metrics for read throughput and duration
-

TR-INGEST-002: Order Data Ingestion from S3

- Related Functional Requirement ID(s): FR-INGEST-002
- Objective:

Read order transaction CSV files from S3 source location into a Spark DataFrame with inferred or explicit schema for downstream processing.

- Source Details:
 - Source Type: S3 CSV Files
 - Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/
 - File Format: CSV
 - Delimiter: Comma (,)
 - Header: Present (first row)
 - Encoding: UTF-8
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - DataFrame Name: df_order_raw
- Input Schema:

	Column Name	Data Type	Nullable	Notes	
	-----	-----	-----	-----	
	OrderId	String	No	Primary key, order identifier	

	ItemName	String	Yes	Product/item name	
	PricePerUnit	Decimal(10,2)	Yes	Unit price	
	Qty	Integer	Yes	Quantity ordered	
	Date	String	Yes	Order date (to be parsed)	
	CustId	String	No	Foreign key to customer	

- Output Schema:

Same as Input Schema (pass-through at this stage)

- Processing / Transformation Logic:

1. Use Spark DataFrame reader to load CSV files

2. Specify options:

- format: "csv"
- header: "true"
- inferSchema: "true" (or explicit schema definition)
- path: s3://adif-sdlc/sdlc_wizard/orderdata/

3. Load all CSV files matching pattern .csv in the directory

4. Store result in df_order_raw DataFrame

5. Log record count: df_order_raw.count()

- Validation Rules:

- Verify S3 path exists and is accessible
- Verify at least one CSV file is present
- Verify header row contains expected column names: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Verify record count > 0 after load
- Verify PricePerUnit and Qty are numeric types

- Error Handling and Logging:

- If S3 path does not exist: Log ERROR "Source path s3://adif-sdlc/sdlc_wizard/orderdata/ not found" and raise exception
- If no CSV files found: Log ERROR "No CSV files found in source path" and raise exception
- If schema mismatch: Log WARNING "Schema mismatch detected, attempting best-effort column mapping" and continue
- If load fails: Log ERROR with exception details and raise exception
- On success: Log INFO "Order data loaded successfully: {record_count} records"

- Runtime Parameters:

- source_order_path: s3://adif-sdlc/sdlc_wizard/orderdata/ (configurable via Glue job parameter)

- Operational Notes:
- Monitor for data type inference issues with PricePerUnit and Qty
- Consider explicit schema definition if inference is unreliable
- Track file count and size metrics in CloudWatch
-

TR-DQ-001: Data Quality Processing for Customer Dataset

- Related Functional Requirement ID(s): FR-DQ-001
- Objective:

Remove null values (both string "Null" and actual NULL) and duplicate records from customer dataset to ensure data quality.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: df_customer_raw (from TR-INGEST-001)
- Target Details:
- Target Type: In-memory Spark DataFrame
- DataFrame Name: df_customer_clean
- Input Schema:

Same as TR-INGEST-001 output schema

- Output Schema:

Same as Input Schema (cleaned subset)

- Processing / Transformation Logic:
1. Start with df_customer_raw
 2. Remove rows with string "Null" (case-sensitive):
 - Filter out rows where any column contains the exact string "Null"

- Use condition: `~(col("CustId") == "Null"   col("Name") == "Null"   col("EmailId") == "Null"   col("Region") == "Null")`
--

3. Remove rows with actual NULL values:
- Use `dropna()` or filter with `isNotNull()` on all columns
4. Remove duplicate rows:
- Use `dropDuplicates()` on all columns
5. Store result in df_customer_clean
6. Calculate and log metrics:

- `initial_count = df_customer_raw.count()`
- `null_string_removed = count after step 2`
- `null_removed = count after step 3`
- `duplicates_removed = count after step 4`
- `final_count = df_customer_clean.count()`
- Validation Rules:
 - Verify `final_count > 0` (critical warning if all records removed)
 - Verify duplicate removal rate `< 50%` (warning if exceeded)
 - Verify null removal is reasonable (warning if `> 30%` removed)
- Error Handling and Logging:
 - If all records removed: Log CRITICAL "All customer records removed during data quality processing" and raise exception
 - If duplicate rate `> 50%`: Log WARNING "High duplicate rate detected: {duplicate_percentage}%"
 - If null rate `> 30%`: Log WARNING "High null rate detected: {null_percentage}%"
 - On success: Log INFO "Customer data quality processing complete: {final_count} records retained, {removed_count} records removed"
- Runtime Parameters:
 - None (uses in-memory DataFrame)
- Operational Notes:
 - Data quality metrics should be published to CloudWatch custom metrics
 - Consider sampling for large datasets to validate quality rules before full processing
-

TR-DQ-002: Data Quality Processing for Order Dataset

- Related Functional Requirement ID(s): FR-DQ-001
- Objective:

Remove null values (both string "Null" and actual NULL) and duplicate records from order dataset to ensure data quality.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: `df_order_raw` (from TR-INGEST-002)
- Target Details:
 - Target Type: In-memory Spark DataFrame

- DataFrame Name: df_order_clean

- Input Schema:

Same as TR-INGEST-002 output schema

- Output Schema:

Same as Input Schema (cleaned subset)

- Processing / Transformation Logic:

1. Start with df_order_raw

2. Remove rows with string "Null" (case-sensitive):

- Filter out rows where any column contains the exact string "Null"

(col("OrderId") == "Null"	col("ItemName") == "Null"	col("PricePerUnit") == "Null"	col("Qty") == "Null"	col("Date") == "Null"	col("C
---------------------------	---------------------------	-------------------------------	----------------------	-----------------------	--------

3. Remove rows with actual NULL values:

- Use `dropna()` or filter with `isNotNull()` on all columns

4. Remove duplicate rows:

- Use `dropDuplicates()` on all columns

5. Store result in df_order_clean

6. Calculate and log metrics:

- initial_count = df_order_raw.count()

- null_string_removed = count after step 2

- null_removed = count after step 3

- duplicates_removed = count after step 4

- final_count = df_order_clean.count()

- Validation Rules:

- Verify final_count > 0 (critical warning if all records removed)

- Verify duplicate removal rate < 50% (warning if exceeded)

- Verify null removal is reasonable (warning if > 30% removed)

- Error Handling and Logging:

- If all records removed: Log CRITICAL "All order records removed during data quality processing" and raise exception

- If duplicate rate > 50%: Log WARNING "High duplicate rate detected: {duplicate_percentage}%"

- If null rate > 30%: Log WARNING "High null rate detected: {null_percentage}%"

- On success: Log INFO "Order data quality processing complete: {final_count} records retained, {removed_count} records removed"

- Runtime Parameters:

- None (uses in-memory DataFrame)
- Operational Notes:
- Data quality metrics should be published to CloudWatch custom metrics
- Monitor for patterns in removed records to identify upstream data issues
-

TR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Register cleaned customer dataset as a queryable table in AWS Glue Data Catalog for downstream analytics and querying via Athena.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: df_customer_clean (from TR-DQ-001)
- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Database Name: gen_ai_poc_databrickscoe
- Table Name: sdlc_wizard_customer
- Storage Location: Not explicitly stored to S3 at this stage (catalog registration only)
- Input Schema:

Same as TR-DQ-001 output schema

- Output Schema:

Glue Catalog table schema matching input:

	Column Name	Data Type	Nullable	
	-----	-----	-----	
	CustId	string	false	
	Name	string	true	
	EmailId	string	true	
	Region	string	true	

- Processing / Transformation Logic:
 1. Verify or create Glue database gen_ai_poc_databrickscoe
 2. Use Glue DynamicFrame or Spark catalog integration to register df_customer_clean
 3. Set table properties:

- classification: "csv" or "parquet" depending on storage format
 - compressionType: "none" or "snappy"
4. Update catalog metadata
 5. Verify table registration by querying Glue Catalog API
- Validation Rules:
 - Verify database exists or is created successfully
 - Verify table schema matches expected columns
 - Verify table is queryable via Athena (optional validation query)
 - Error Handling and Logging:
 - If database creation fails: Log ERROR "Failed to create database gen_ai_poc_databrickscoe" and raise exception
 - If table registration fails: Log ERROR "Failed to register table sdhc_wizard_customer" and raise exception
 - On success: Log INFO "Customer table registered successfully in Glue Catalog: gen_ai_poc_databrickscoe.sdhc_wizard_customer"
 - Runtime Parameters:
 - glue_database: gen_ai_poc_databrickscoe (configurable)
 - glue_table_customer: sdhc_wizard_customer (configurable)
 - Operational Notes:
 - Table registration does not persist data to S3 unless explicitly written
 - Consider writing cleaned data to S3 before catalog registration for persistence
 - Use Glue Crawler as alternative registration method if preferred
 -

TR-CATALOG-002: Register Order Table in Glue Data Catalog

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Register cleaned order dataset as a queryable table in AWS Glue Data Catalog for downstream analytics and querying via Athena.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: df_order_clean (from TR-DQ-002)
- Target Details:

- Target Type: AWS Glue Data Catalog Table
- Database Name: gen_ai_poc_databrickscoee
- Table Name: sdlc_wizard_order
- Storage Location: Not explicitly stored to S3 at this stage (catalog registration only)

- Input Schema:

Same as TR-DQ-002 output schema

- Output Schema:

Glue Catalog table schema matching input:

	Column Name	Data Type	Nullable	
	-----	-----	-----	
	OrderId	string	false	
	ItemName	string	true	
	PricePerUnit	decimal(10,2)	true	
	Qty	int	true	
	Date	string	true	
	CustId	string	false	

- Processing / Transformation Logic:

1. Verify database gen_ai_poc_databrickscoee exists (created in TR-CATALOG-001)
2. Use Glue DynamicFrame or Spark catalog integration to register df_order_clean
3. Set table properties:
 - classification: "csv" or "parquet" depending on storage format
 - compressionType: "none" or "snappy"
4. Update catalog metadata
5. Verify table registration by querying Glue Catalog API

- Validation Rules:

- Verify database exists
- Verify table schema matches expected columns
- Verify table is queryable via Athena (optional validation query)

- Error Handling and Logging:

- If table registration fails: Log ERROR "Failed to register table sdlc_wizard_order" and raise exception
- On success: Log INFO "Order table registered successfully in Glue Catalog: gen_ai_poc_databrickscoee.sdlc_wizard_order"

- Runtime Parameters:

- glue_database: gen_ai_poc_databrickscoe (configurable)
- glue_table_order: sdlc_wizard_order (configurable)
- Operational Notes:
 - Table registration does not persist data to S3 unless explicitly written
 - Consider writing cleaned data to S3 before catalog registration for persistence
-

TR-SCD2-001: Implement SCD Type 2 for Order Summary Using Apache Hudi

- Related Functional Requirement ID(s): FR-SCD2-001, FR-INCR-001
- Objective:

Join customer and order data, implement Slowly Changing Dimension Type 2 logic using Apache Hudi to track historical changes, and store results in S3 with Glue Catalog registration.

- Source Details:
 - Source Type 1: In-memory Spark DataFrame
 - Source Name 1: df_customer_clean (from TR-DQ-001)
 - Source Type 2: In-memory Spark DataFrame
 - Source Name 2: df_order_clean (from TR-DQ-002)
 - Source Type 3: Hudi Table (for incremental runs)
 - Source Path 3: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Details:
 - Target Type: Apache Hudi Table (Copy-on-Write)
 - Target Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - Glue Catalog Database: gen_ai_poc_databrickscoe
 - Glue Catalog Table: ordersummary
 - Hudi Table Type: COPY_ON_WRITE
 - Hudi Operation: UPSERT
- Input Schema:

Customer (df_customer_clean):

	Column Name	Data Type	Nullable	
	-----	-----	-----	
	CustId	string	false	
	Name	string	true	
	EmailId	string	true	

	Region	string	true	
--	--------	--------	------	--

Order (df_order_clean):

	Column Name	Data Type	Nullable	
	-----	-----	-----	
	OrderId	string	false	
	ItemName	string	true	
	PricePerUnit	decimal(10,2)	true	
	Qty	int	true	
	Date	string	true	
	CustId	string	false	

- Output Schema:

Hudi Table ordersummary:

	Column Name	Data Type	Nullable	Notes	
	-----	-----	-----	-----	
	CustId	string	false	From customer/order join	
	OrderId	string	false	From order	
	Name	string	true	From customer	
	EmailId	string	true	From customer	
	Region	string	true	From customer	
	ItemName	string	true	From order	
	PricePerUnit	decimal(10,2)	true	From order	
	Qty	int	true	From order	
	Date	string	true	From order	
	IsActive	boolean	false	SCD2 flag: true for current, false for historical	
	StartDate	date	false	SCD2 effective start date	
	EndDate	date	true	SCD2 effective end date (null for active)	
	OpTs	timestamp	false	Operation timestamp (job execution time)	

- Processing / Transformation Logic:

- Initial Load (First Run):

1. Join df_customer_clean and df_order_clean on CustId (inner join)
2. Add SCD2 columns:

- IsActive = true
- StartDate = current_date()
- EndDate = null
- OpTs = current_timestamp()

3. Configure Hudi options:

- `hoodie.table.name = "ordersummary"`
- `hoodie.datasource.write.recordkey.field = "CustId,OrderId"`
- `hoodie.datasource.write.precombine.field = "OpTs"`
- `hoodie.datasource.write.partitionpath.field = ""` (no partitioning specified)
- `hoodie.datasource.write.table.type = "COPY_ON_WRITE"`
- `hoodie.datasource.write.operation = "insert"`
- `hoodie.datasource.write.hive_style_partitioning = "false"`

4. Write to `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`

5. Register in Glue Catalog as `gen_ai_poc_databricksco.ordersummary`

• Incremental Load (Subsequent Runs):

1. Read existing Hudi table from `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`

2. Filter for active records: `IsActive = true`

3. Join `df_customer_clean` and `df_order_clean` on `CustId` (inner join) to create `df_incoming`

4. Compare `df_incoming` with active records from Hudi table:

- Compare business columns: `Name`, `EmailId`, `Region`, `ItemName`, `PricePerUnit`, `Qty`, `Date`
- Identify changed records (any business column differs)
- Identify new records (`CustId+OrderId` not in existing table)
- Identify unchanged records (all business columns match)

5. For changed records:

- Create "close" records: copy existing active record, set `IsActive = false`, set `EndDate = current_date()`, keep original `StartDate` and `OpTs`
- Create "new" records: copy incoming record, set `IsActive = true`, set `StartDate = current_date()`, set `EndDate = null`, set `OpTs = current_timestamp()`

6. For new records:

- Create records with `IsActive = true`, `StartDate = current_date()`, `EndDate = null`, `OpTs = current_timestamp()`

7. For unchanged records:

- No action (do not write)

8. Combine "close" records and "new" records into `df_upsert`

9. Configure Hudi options:

- `hoodie.datasource.write.operation = "upsert"`
- Other options same as initial load

10. Write `df_upsert` to Hudi table using upsert operation

11. Update Glue Catalog metadata if schema changed

- Validation Rules:
 - Verify join produces records (warning if zero)
 - Verify no duplicate active records for same CustId+OrderId combination
 - Verify all active records have EndDate = null
 - Verify all closed records have EndDate <= current_date()
 - Verify OpTs is populated for all records
- Error Handling and Logging:
 - If join produces zero records: Log WARNING "Customer-Order join produced no records"
 - If Hudi write fails: Log ERROR "Hudi write operation failed" with exception details and raise exception
 - If multiple active records detected: Log CRITICAL "Data integrity violation: multiple active records for key {CustId}+{OrderId}" and raise exception
 - On success: Log INFO "SCD2 ordersummary processing complete: {insert_count} inserts, {update_count} updates"
- Runtime Parameters:
 - target_ordersummary_path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (configurable)
 - glue_database: gen_ai_poc_databrickscoe (configurable)
 - glue_table_ordersummary: ordersummary (configurable)
 - hudi_table_name: ordersummary (configurable)
 - hudi_record_key: "CustId,OrderId" (configurable)
 - hudi_precombine_field: "OpTs" (configurable)
- Operational Notes:
 - Hudi library must be available in Glue job classpath
 - Use Glue 3.0+ with --datalake-formats hudi option
 - Monitor Hudi commit timeline for compaction and cleaning operations
 - Consider partitioning by Date or Region for large datasets (not specified in FRD)
 - SCD2 logic assumes full dataset comparison; optimize with change data capture if available
-

TR-AGG-001: Generate Customer Aggregate Spend Table

- Related Functional Requirement ID(s): FR-AGG-001
- Objective:

Aggregate order data by customer to calculate total spending and store results in S3 with Glue Catalog registration for business intelligence purposes.

- Source Details:
- Source Type 1: In-memory Spark DataFrame
- Source Name 1: df_customer_clean (from TR-DQ-001)
- Source Type 2: In-memory Spark DataFrame
- Source Name 2: df_order_clean (from TR-DQ-002)
- Target Details:
- Target Type: S3 Parquet Files
- Target Path: s3://adif-sdlc/analytics/customeraggregatespend/
- Glue Catalog Database: gen_ai_poc_databricksco
- Glue Catalog Table: customeraggregatespend
- Write Mode: Overwrite
- Input Schema:

Customer (df_customer_clean):

	Column Name	Data Type	Nullable	
	-----	-----	-----	
	CustId	string	false	
	Name	string	true	
	EmailId	string	true	
	Region	string	true	

Order (df_order_clean):

	Column Name	Data Type	Nullable	
	-----	-----	-----	
	OrderId	string	false	
	ItemName	string	true	
	PricePerUnit	decimal(10,2)	true	
	Qty	int	true	
	Date	string	true	
	CustId	string	false	

- Output Schema:

	Column Name	Data Type	Nullable	Derivation	
	-----	-----	-----	-----	
	Name	string	true	Direct map from customer.Name	
	TotalAmount	decimal(20,2)	true	SUM(PricePerUnit * Qty) grouped by customer	
	Date	string	true	Direct map from order.Date (latest or all dates - not specified)	

- Processing / Transformation Logic:
 1. Join df_customer_clean and df_order_clean on CustId (inner join)
 2. Calculate TotalAmount per customer:
 - Create calculated column: TotalAmount = PricePerUnit Qty
 - Group by CustId, Name
 - Aggregate: SUM(TotalAmount) as TotalAmount
 3. Include Date in output (assumption: use latest Date per customer or include all dates)
 4. Select final columns: Name, TotalAmount, Date
 5. Write to s3://adif-sdlc/analytics/customeraggregatespend/ in Parquet format
 6. Write mode: overwrite (full refresh)
 7. Register in Glue Catalog as gen_ai_poc_databrickscoe.customeraggregatespend
- Validation Rules:
 - Verify aggregation produces records (warning if zero)
 - Verify TotalAmount >= 0 (warning if negative values found)
 - Verify no null values in Name or TotalAmount after aggregation
- Error Handling and Logging:
 - If aggregation produces zero records: Log WARNING "Customer aggregate spend calculation produced no records"
 - If negative TotalAmount detected: Log WARNING "Negative TotalAmount detected for customer {Name}: {TotalAmount}"
 - If write operation fails: Log ERROR "Failed to write customer aggregate spend table" with exception details and raise exception
 - On success: Log INFO "Customer aggregate spend table created successfully: {record_count} customers"
- Runtime Parameters:
 - target_aggregate_path: s3://adif-sdlc/analytics/customeraggregatespend/ (configurable)
 - glue_database: gen_ai_poc_databrickscoe (configurable)
 - glue_table_aggregate: customeraggregatespend (configurable)
- Operational Notes:
 - Aggregation logic assumes one TotalAmount per customer (sum across all orders)
 - Date field handling is ambiguous (latest date vs. all dates) - implementation should clarify
 - Consider partitioning by Date or Region for query optimization (not specified in FRD)
 - Use Parquet format for efficient columnar storage and compression
 -

3. Runtime Strategy Notes

Authoring Language

- Primary Language: Python 3.7+
- Framework: PySpark (AWS Glue Spark runtime)
- Libraries:
- `pyspark.sql` for DataFrame operations
- `aws glue.context` for Glue-specific operations
- `aws glue.dynamicframe` for Glue DynamicFrame operations
- `hudi` library for SCD2 implementation (version 0.10.0+)

Execution Strategy

- Job Type: AWS Glue ETL Job (Spark)
- Glue Version: 3.0 or higher
- Worker Type: G.1X or G.2X (recommended: G.1X for development, G.2X for production)
- Number of Workers: Minimum 2, scale based on data volume
- Job Timeout: 2880 minutes (48 hours) default, adjust based on data volume
- Max Retries: 0 (handle retries at orchestration layer)
- Execution Mode: Standard (batch processing)
- Scheduling: Triggered by AWS Glue workflow, Step Functions, or EventBridge schedule

Glue Job Type Expectations

- Job Type: Spark (not Streaming or Python Shell)
- Script Location: `s3:///scripts/customer_order_pipeline.py`
- Temporary Directory: `s3:///temp/`
- Spark UI Logs: `s3:///sparkHistoryLogs/`
- Enable Job Metrics: Yes (publish to CloudWatch)
- Enable Continuous Logging: Yes (CloudWatch Logs)
- Enable Spark UI: Yes (for debugging)
- Additional Python Modules: None required beyond Glue defaults
- Additional Jars: Hudi bundle jar (provided by `--datalake-formats hudi` option)
- Job Parameters:
- `datalake-formats: hudi`
- `enable-glue-datacatalog: true`
- `job-language: python`
- `TempDir: s3:///temp/`

- enable-metrics: true
- enable-continuous-cloudwatch-log: true
-

4. Data Design Details

Source Paths / Source Systems

1. Customer Data:

- Path: s3://adif-sdlc/sdlc_wizard/customerdata/
- Format: CSV
- Pattern: .csv
- Encoding: UTF-8

2. Order Data:

- Path: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Pattern: .csv
- Encoding: UTF-8

3. Existing Order Summary (Incremental Runs):

- Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Format: Apache Hudi (COPY_ON_WRITE)

Target Paths / Target Tables

1. Order Summary (SCD2):

- Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Format: Apache Hudi (COPY_ON_WRITE)
- Catalog: gen_ai_poc_databricksco.ordersummary

2. Customer Aggregate Spend:

- Path: s3://adif-sdlc/analytics/customeraggregatespend/
- Format: Parquet
- Catalog: gen_ai_poc_databricksco.customeraggregatespend

Catalog Registration Expectations

- Database: gen_ai_poc_databricksco (create if not exists)
- Tables:
 1. sdlc_wizard_customer (optional persistence, catalog registration only)
 2. sdlc_wizard_order (optional persistence, catalog registration only)

3. ordersummary (Hudi table with automatic catalog sync)
 4. customeraggregatespend (Parquet table with manual catalog registration)
- Catalog Sync: Use Glue's built-in Hudi catalog sync for ordersummary
 - Crawler: Not required if catalog registration is handled programmatically

Partitioning Expectations

- Customer Data: No partitioning specified
- Order Data: No partitioning specified
- Order Summary: No partitioning specified (consider Date or Region partitioning for optimization)
- Customer Aggregate Spend: No partitioning specified (consider Date or Region partitioning for optimization)

File Formats / Table Formats

- Source: CSV (comma-delimited, UTF-8, with header)
- Curated Layer: Apache Hudi (COPY_ON_WRITE)
- Analytics Layer: Parquet (Snappy compression recommended)

Update / Overwrite / Append Behavior

1. Customer and Order Ingestion: Read-only (no write to source)
2. Order Summary (SCD2):
 - Initial Load: INSERT operation
 - Incremental Load: UPSERT operation (Hudi handles insert/update logic)
 - Behavior: Append new versions, update IsActive/EndDate for changed records
3. Customer Aggregate Spend:
 - Write Mode: OVERWRITE (full refresh on each run)
 - Behavior: Replace entire table with new aggregated data

Important Note on Source Fidelity

- No staging tables or intermediate sources are introduced.
- All processing uses the original sources specified:
 - s3://adif-sdlc/sdlc_wizard/customerdata/ for customer data
 - s3://adif-sdlc/sdlc_wizard/orderdata/ for order data
- Cleaned DataFrames (df_customer_clean, df_order_clean) are in-memory transformations, not persisted intermediate sources.
-

5. Processing Details

Data Quality Rules

1. Null String Removal:

- Remove rows where any column contains the exact string "Null" (case-sensitive)
- Apply to all columns in customer and order datasets

2. Actual NULL Removal:

- Remove rows where any column contains actual NULL values
- Use Spark's ``dropna()`` or ``isNotNull()`` filter

3. Duplicate Removal:

- Remove duplicate rows based on exact match across all columns
- Use Spark's ``dropDuplicates()`` without specifying subset (all columns)

4. Validation Thresholds:

- Critical: If all records removed, halt processing
- Warning: If duplicate rate > 50%, log warning
- Warning: If null removal rate > 30%, log warning

Deduplication Rules

- Scope: Apply to entire row (all columns)
- Method: Exact match comparison
- Retention: Keep first occurrence (Spark default behavior)
- Logging: Log count of duplicates removed per dataset

Join Rules

1. Customer-Order Join (for SCD2 and Aggregation):

- Join Type: INNER JOIN
- Join Key: CustId
- Result: Only customers with orders are included
- Null Handling: Join key must be non-null (enforced by data quality rules)

2. SCD2 Comparison Join (Incremental):

- Join Type: LEFT OUTER JOIN (incoming data left, existing active records right)
- Join Key: CustId + OrderId
- Purpose: Identify new, changed, and unchanged records

Aggregation Rules

1. Customer Aggregate Spend:

- Group By: CustId, Name
- Aggregation: SUM(PricePerUnit Qty) as TotalAmount

- Null Handling: Exclude rows with null PricePerUnit or Qty before aggregation
- Output: One row per customer with total spend across all orders

SCD / History Tracking Rules

1. SCD Type 2 Implementation:

- Track changes to: Name, EmailId, Region, ItemName, PricePerUnit, Qty, Date
- Primary Key: CustId + OrderId (composite)
- Versioning Columns:
 - IsActive: boolean (true = current version, false = historical version)
 - StartDate: date (effective start date of this version)
 - EndDate: date (effective end date of this version, null for active)
 - OpTs: timestamp (operation timestamp for precombine logic)

2. Change Detection:

- Compare all business columns between incoming and existing active records
- If any column differs, treat as changed record
- If CustId+OrderId not in existing table, treat as new record
- If all columns match, treat as unchanged (no action)

3. Version Management:

- On change: Close existing active record (IsActive=false, EndDate=current_date), insert new active record (IsActive=true, StartDate=current_date, EndDate=null)
- On new: Insert new active record (IsActive=true, StartDate=current_date, EndDate=null)
- On unchanged: No action (do not write)

4. Hudi Configuration:

- Record Key: CustId, OrderId
- Precombine Field: OpTs (use latest timestamp for conflict resolution)
- Table Type: COPY_ON_WRITE (optimized for read-heavy workloads)
- Operation: INSERT (initial), UPSERT (incremental)

Null Handling

1. Source Data:

- Remove rows with string "Null" (case-sensitive)
- Remove rows with actual NULL values
- Apply before any transformations

2. Derived Columns:

- IsActive: Never null (default true for new records)

- StartDate: Never null (default current_date())
- EndDate: Null for active records, populated date for closed records
- OpTs: Never null (default current_timestamp())
- TotalAmount: Null if PricePerUnit or Qty is null (excluded from aggregation)

Type Casting Expectations

1. Customer Data:
- CustId: Cast to string if not already
 - Name: Cast to string
 - EmailId: Cast to string
 - Region: Cast to string
2. Order Data:
- OrderId: Cast to string if not already
 - ItemName: Cast to string
 - PricePerUnit: Cast to decimal(10,2)
 - Qty: Cast to integer
 - Date: Cast to string initially, parse to date type if needed for SCD2 logic
 - CustId: Cast to string if not already
3. SCD2 Columns:
- IsActive: boolean
 - StartDate: date (YYYY-MM-DD)
 - EndDate: date (YYYY-MM-DD) or null
 - OpTs: timestamp (YYYY-MM-DD HH:MM:SS)
4. Aggregation Columns:
- TotalAmount: decimal(20,2) (sufficient for large sums)
 -

6. Traceability Matrix

(s)	Original Source	Technical Source	Descri
	-----	-----	-----
	s3://adif-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdlc_wizard/customerdata/	Custom
	s3://adif-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdlc_wizard/orderdata/	Order
002	df_customer_raw, df_order_raw	df_customer_raw, df_order_raw	Data q
TR-CATALOG-002	df_customer_clean, df_order_clean	df_customer_clean, df_order_clean	Glue C

	df_customer_clean, df_order_clean	df_customer_clean, df_order_clean	SCD T
	df_customer_clean, existing ordersummary	df_customer_clean, s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Increm
	df_customer_clean, df_order_clean	df_customer_clean, df_order_clean	Custon

- Source Fidelity Verification:
- All technical sources match the original sources specified in the FRD
- No intermediate staging tables or alternative sources have been introduced
- In-memory DataFrames (df_customer_raw, df_customer_clean, df_order_raw, df_order_clean) are transformations of the original sources, not separate source systems
-

7. ETL Mapping Requirements

Customer Data Mapping

- Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (CSV)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Notes	
	-----	-----	-----	-----	-----	
	CustId	string	Variable	No	Primary key	
	Name	string	Variable	Yes	Customer full name	
	EmailId	string	Variable	Yes	Email address	
	Region	string	Variable	Yes	Geographic region	

- Target: df_customer_clean (In-Memory DataFrame)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	CustId	string	Variable	No	Direct map from source.CustId	
	Name	string	Variable	Yes	Direct map from source.Name	
	EmailId	string	Variable	Yes	Direct map from source.EmailId	
	Region	string	Variable	Yes	Direct map from source.Region	

- Transformation: Data quality processing (null removal, deduplication)
-

Order Data Mapping

- Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (CSV)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Notes	
	-----	-----	-----	-----	-----	

	OrderId	string	Variable	No	Primary key	
	ItemName	string	Variable	Yes	Product name	
	PricePerUnit	decimal	10,2	Yes	Unit price	
	Qty	integer	-	Yes	Quantity	
	Date	string	Variable	Yes	Order date	
	CustId	string	Variable	No	Foreign key to customer	

- Target: df_order_clean (In-Memory DataFrame)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	OrderId	string	Variable	No	Direct map from source.OrderId	
	ItemName	string	Variable	Yes	Direct map from source.ItemName	
	PricePerUnit	decimal	10,2	Yes	Direct map from source.PricePerUnit	
	Qty	integer	-	Yes	Direct map from source.Qty	
	Date	string	Variable	Yes	Direct map from source.Date	
	CustId	string	Variable	No	Direct map from source.CustId	

- Transformation: Data quality processing (null removal, deduplication)

•

Order Summary (SCD2) Mapping

- Source 1: df_customer_clean (In-Memory DataFrame)
- Source 2: df_order_clean (In-Memory DataFrame)
- Target: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (Hudi Table)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	CustId	string	Variable	No	Direct map from df_customer_clean.CustId (join key)	
	OrderId	string	Variable	No	Direct map from df_order_clean.OrderId	
	Name	string	Variable	Yes	Direct map from df_customer_clean.Name	
	EmailId	string	Variable	Yes	Direct map from df_customer_clean.EmailId	
	Region	string	Variable	Yes	Direct map from df_customer_clean.Region	
	ItemName	string	Variable	Yes	Direct map from df_order_clean.ItemName	
	PricePerUnit	decimal	10,2	Yes	Direct map from df_order_clean.PricePerUnit	
	Qty	integer	-	Yes	Direct map from df_order_clean.Qty	
	Date	string	Variable	Yes	Direct map from df_order_clean.Date	
	IsActive	boolean	-	No	Derived: true for active records, false for historical	
	StartDate	date	-	No	Derived: current_date() for new/changed records	

	EndDate	date	-	Yes	Derived: null for active, current_date() for closed records	
	OpTs	timestamp	-	No	Derived: current_timestamp() at processing time	

- Transformation: Inner join on CustId, SCD Type 2 logic (version management)
-

Customer Aggregate Spend Mapping

- Source 1: df_customer_clean (In-Memory DataFrame)
- Source 2: df_order_clean (In-Memory DataFrame)
- Target: s3://adif-sdlc/analytics/customeraggregatespend/ (Parquet)

Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping
-----	-----	-----	-----	-----
Name	string	Variable	Yes	Direct map from df_customer_clean.Name
TotalAmount	decimal	20,2	Yes	Derived: SUM(df_order_clean.PricePerUnit * df_order_clean.Qty) grouped by customer
Date	string	Variable	Yes	Direct map from df_order_clean.Date (ambiguous: latest or all dates)

- Transformation: Inner join on CustId, aggregation (SUM), grouping by CustId and Name
-

Assumptions

1. Date Field Handling in Customer Aggregate Spend:
 - The FRD specifies including "Date" in the customeraggregatespend output but does not clarify whether this should be:
 - The latest order date per customer
 - All order dates (resulting in multiple rows per customer)
 - A specific date (e.g., processing date)
 - Assumption: Implementation will use the latest order date per customer (MAX(Date) in aggregation)
2. Data Type Precision:
 - Exact data types for CustId, OrderId, Name, EmailId, Region, ItemName are not specified in FRD
 - Assumption: All are treated as string type with variable length
3. Date Format:
 - Date field format in order data is not specified
 - Assumption: Date is in ISO 8601 format (YYYY-MM-DD) or parseable string format
4. Hudi Precombine Logic:

- FRD does not specify conflict resolution strategy for concurrent updates
- Assumption: OpTs (operation timestamp) is used as precombine field; latest timestamp wins

5. Partitioning Strategy:

- No partitioning strategy is specified in FRD
- Assumption: No partitioning is implemented; can be added for optimization if needed

6. Incremental Processing Trigger:

- FRD mentions incremental processing based on customer updates but does not specify change detection mechanism
- Assumption: Full dataset comparison is performed on each run (no CDC or watermark-based incremental logic)

7. Glue Catalog Persistence:

- FRD specifies catalog registration for customer and order tables but does not specify S3 persistence
- Assumption: Cleaned data may or may not be persisted to S3; catalog registration can be done on in-memory DataFrames or persisted data

8. Error Recovery:

- FRD does not specify error recovery or retry logic
- Assumption: Job fails on critical errors; retry logic is handled at orchestration layer (Step Functions, Glue Workflow)

9. Hudi Compaction and Cleaning:

- FRD does not specify Hudi table maintenance operations
- Assumption: Hudi's default compaction and cleaning schedules are used; manual tuning may be required for production

10. CloudWatch Metrics:

- FRD mentions logging to CloudWatch but does not specify custom metrics
- Assumption: Standard Glue job metrics are used; custom metrics for data quality can be added

•

- Note: Not enough information available in the FRD to derive the following details:
- Exact data type lengths/precision for string fields (CustId, OrderId, Name, EmailId, Region, ItemName)
- Specific date format for Date field in order data
- Partitioning strategy for Hudi and Parquet tables
- Incremental processing change detection mechanism (CDC, watermark, full comparison)
- Date field handling in customeraggregatespend (latest date vs. all dates)

- Hudi compaction and cleaning configuration
- Custom CloudWatch metrics definitions
-
- Document Version: 1.0
- Date: Generated from FRD
- Status: Ready for Implementation